WebBeans Authentication Guide

2026-10-28

Contents

1. Principles

2. Identity model

3. Credentials

4. The provider SPI

4.1 Redirect flows
4.2 Token flows (browser SDKs)
4.3 Infrastructure identity

5. The login page

6. Roles and authorization

7. Authenticator (the platform-side service)

8. Session policy and events

8a. Session store (sharing sessions between instances)

9. HTTP surface (fixed, provider-independent)

10. Providers shipped with the platform

11. Examples

12. Security notes

13. Implementation plan

14. Load balancing and multi-instance deployment

v0.2 (step 18): §8a `SessionStore` — sessions shared between instances — and §14 load balancing and multi-instance deployment.

Scope: how a user becomes a `Principal` in a `UISession`, how applications plug in their own identity systems (Firebase, an existing SSO, a home-grown user table, a reverse proxy, LDAP, OAuth2/OIDC), and how the login page, session policy and authorization hook into that. Package `org.webbeans.auth` in a new module `webbeans-auth` (API + shipped providers); the server (step 2) depends on it. JDK 8, no third-party dependencies in the API or the shipped providers (JWT verification uses `java.security`).

1. Principles

- **The platform owns sessions; providers own identity.** A provider turns *credentials* into a `Principal`. Everything after that — the cookie, the `UISession`, timeouts, logout, CSRF — is the platform's and identical for every provider, so switching from a properties file to Firebase changes one registration, not the application.
- **Three flow shapes cover every real system**: *credentials posted to the server* (form login, API key, proxy headers), *redirect to an identity provider and back* (OAuth2/OIDC, SAML-style), and *token obtained in the browser* (Firebase, Auth0, Cognito JS SDKs — the client signs in, the server verifies the token). The API models the shapes, not the vendors.
- **Roles are the application's currency.** Providers deliver claims/groups; a `RoleMapper` turns them into the role names the registry filters on (`roles = {"admin"}`). Fine-grained checks go through an `Authorizer`.
- **Everything is a Lookup service** (`@ServiceProvider`), ordered by `position`, replaceable per application, testable with `MockServices`.

2. Identity model

```
public final class Principal {                          // immutable;
Principal.builder("alice")...build()
    String getId();                                     // stable unique id from the provider (uid, sub,
DN, username)
    String getName();                                   // login name shown in logs; may equal id
    String getDisplayName();
    String getEmail();                                  // optional
    Set<String> getRoles();                             // application roles, after RoleMapper
    Map<String, String> getAttributes();                // provider claims worth keeping (picture,
tenant, department...)
    String getProviderId();                             // which provider authenticated this principal
    boolean hasRole(String role);  boolean hasAnyRole(String... roles);
}

public final class Authentication {                     // what the session keeps about how the principal
got in
    Principal getPrincipal();
    Instant getTime();                                  // when authenticated
    Instant getExpiresAt();                             // null = until session end; set by token
providers
    Set<String> getMethods();                           // "password", "oidc", "mfa:totp"... for step-up
decisions
    Map<String, Object> getSecrets();                   // server-side only: access/refresh tokens for
calling the provider's APIs; never serialised
}
```

`UISession.getPrincipal()` and `UISession.getAuthentication()` expose these; `Principal` is also in the session Lookup.

3. Credentials

Credentials are what arrives at the server. They are plain value objects; a provider declares which it accepts.

```
public interface Credentials { }

public final class PasswordCredentials implements Credentials { String getUsername(); char[]
getPassword(); void clear(); }
public final class TokenCredentials implements Credentials {               // a bearer token the
client obtained elsewhere
    String getScheme();                                                      // "firebase", "jwt",
"apikey", "auth0"...
    String getToken();
}
public final class HeaderCredentials implements Credentials {              // identity asserted by
infrastructure
    Map<String, String> getHeaders();  String getRemoteAddress();  boolean isFromTrustedProxy();
}
public final class CallbackCredentials implements Credentials {            // the return leg of a
redirect flow
    String getProviderId();  Map<String, String> getParameters();  String getState();  String
getRedirectUri();
}
public final class ChallengeResponse implements Credentials {              // answer to a multi-step
challenge (MFA code...)
    String getChallengeId();  Map<String, String> getValues();
}
```

Applications may define their own `Credentials` classes (a smart-card assertion, a signed launch ticket) and a provider that accepts them; the platform never inspects credential contents.

4. The provider SPI

```
public interface AuthenticationProvider {
    String getId();                                                  // "local", "google", "firebase",
"corp-sso"
    String getDisplayName();                                         // "#CTL_SignInWithGoogle"
allowed
    boolean supports(Class<? extends Credentials> type);
    AuthenticationResult authenticate(Credentials credentials, AuthContext context) throws
AuthenticationException;

    /** How the login page offers this provider; null = not offered (programmatic / infrastructure
providers). */
    default LoginMethod getLoginMethod() { return null; }

    /** Called by the session policy while a session lives; return false to end the session (user
disabled, token revoked). */
    default boolean revalidate(Authentication auth, AuthContext context) { return true; }

    /** Hook for logout (OIDC end-session, token revocation). */
    default void logout(Authentication auth, AuthContext context) { }
}

public final class AuthenticationResult {
    static AuthenticationResult success(Principal p);                        // +
.expiresAt(Instant).method("password").secret("accessToken", t)
    static AuthenticationResult failure(String messageKey);                  // "#ERR_BadPassword";
never reveals which part was wrong
    static AuthenticationResult challenge(Challenge c);                      // multi-step: show
another step before success
    static AuthenticationResult notMine();                                   // let the next provider
try (e.g. unknown realm)
    Kind getKind();  Principal getPrincipal();  Challenge getChallenge();  String getMessage(); ...
}

public final class Challenge {                                               // second factor, consent
screen, password change
    String getId();  String getType();  List<LoginField> getFields();  Map<String, Object>
getClientData();  Instant getExpiresAt();
}

public interface AuthContext {                                               // what a provider may
know about the request
    String getRemoteAddress();  String getUserAgent();  Locale getLocale();
    Optional<String> getHeader(String name);  Optional<String> getParameter(String name);
    Map<String, Object> getFlowState();                                      // survives a redirect
round trip (OAuth state, PKCE verifier)
    UISession getExistingSession();                                          // non-null when re-
authenticating / linking
}

public class AuthenticationException extends Exception { }                   // infrastructure failure
(LDAP down), not a wrong password
```

Registration: `@ServiceProvider(service = AuthenticationProvider.class, position = 100)`. The `Authenticator` (§7) asks providers in position order; the first that `supports` the credential class and does not answer `notMine()` decides.

4.1 Redirect flows

```
public interface RedirectAuthenticationProvider extends AuthenticationProvider {
    /** Build the URL to send the browser to; store anything needed later in context.getFlowState().
*/
    URI start(AuthContext context, URI callbackUri);
    /** Receives CallbackCredentials; typically exchanges the code for tokens and maps claims. */
    // authenticate(CallbackCredentials, ctx) — inherited
}
```

The platform wires `GET /auth/<id>/start` → `start(...)` → 302, and `GET|POST /auth/<id>/callback` → `authenticate(new CallbackCredentials(...))`. Flow state is kept server-side keyed by the `state` parameter (single use, 10-minute expiry), which also defeats CSRF on the callback.

4.2 Token flows (browser SDKs)

```
public interface TokenAuthenticationProvider extends AuthenticationProvider {
    String getScheme();                                                  // matches
TokenCredentials.getScheme()
    // authenticate(TokenCredentials, ctx): verify signature, issuer, audience, expiry; map claims
}
```

The login page runs the provider's `LoginMethod.script(...)`; the script obtains a token with the vendor SDK and posts it to `POST /auth/token` (`{"scheme": "firebase", "token": "..."}`) — the platform creates the session and answers with the redirect target. Token refresh is `POST /auth/token` again on an existing session: the provider's result replaces the `Authentication` without recreating the session.

4.3 Infrastructure identity

`HeaderCredentials` are built by the platform from every request when a `HeaderAuthenticationProvider` is registered and the request comes from a trusted proxy (`--trusted-proxies`); a session is created silently on first contact, so a reverse proxy doing SAML/Kerberos upstream just works.

5. The login page

```
public abstract class LoginMethod {
    public static LoginMethod form(LoginField... fields);                  // username/password by
default
    public static LoginMethod redirect(String buttonLabel, Resource icon);  // "Sign in with Google" →
/auth/<id>/start
    public static LoginMethod script(Resource js, Map<String, Object> config, String...
externalScripts);  // Firebase UI etc.
}
public final class LoginField { String getName(); String getLabel(); Kind getKind(); /* TEXT,
PASSWORD, CHOICE, HIDDEN, CHECKBOX */ boolean isRequired(); List<String> getChoices(); }

public interface LoginPage {                                               // replaceable:
@ServiceProvider(service = LoginPage.class)
    void render(LoginPageModel model, LoginPageOutput out);                 // model: methods, error
message, locale, branding, csrf token, return URL
}
```

The default `LoginPage` renders every offered method: forms as forms, redirect methods as buttons, script methods by loading the script with its config (vendor SDKs from the configured allowlist of script origins, see `--csp-script-src`). A `Challenge` renders as a second page with the challenge's fields. Applications replace the page for branding or write one in their own stack; the contract is only the endpoints in §9.

6. Roles and authorization

```
public interface RoleMapper {                                              // optional; chain by
position
    Set<String> mapRoles(Principal raw, Map<String, Object> claims, AuthenticationProvider source);
}
public interface Authorizer {                                              // optional; default =
role check
    boolean isAllowed(Principal principal, String permission, Object target);
}
public final class Security {
    static Principal currentPrincipal();                                   //
UISession.current().getPrincipal()
    static boolean hasRole(String role);
    static boolean isAllowed(String permission, Object target);            // asks every Authorizer;
all must agree
    static void checkRole(String role) throws AccessDeniedException;
}
```

Registry `roles` filtering (step 1) and `@…Registration(roles = …)` use `Principal.getRoles()`; `Security.isAllowed` is for data-level checks inside actions and services. A `RoleMapper` is where "Firebase custom claim `admin: true` → role `admin`" or "LDAP group `cn=finance` → role `accountant`" lives, so providers stay generic.

7. Authenticator (the platform-side service)

```
public interface Authenticator {                                                    // in Lookup.getDefault();
used by the server handlers and by custom endpoints
    LoginOutcome login(Credentials c, AuthContext ctx);                             // runs providers,
listeners, throttle, creates the session
    void logout(UISession session, String reason);
    LoginOutcome reauthenticate(UISession session, Credentials c, AuthContext ctx);   // step-up /
refresh; keeps the session
    List<AuthenticationProvider> getProviders();
}
public final class LoginOutcome { enum Kind { SESSION, CHALLENGE, FAILURE } UISession getSession();
Challenge getChallenge(); String getMessage(); }
```

Behaviour the platform guarantees around every provider: session fixation protection (new session id on login), `HttpOnly; Secure; SameSite=Lax` cookie, CSRF token on form and JSON posts, per-IP and per-account throttling (`LoginThrottle` SPI, default 10 failures / 5 min), audit through listeners, credentials cleared after use, nothing secret in logs.

8. Session policy and events

```
public interface SessionPolicy {                                        // one, replaceable
    Duration idleTimeout(Principal p);                                  // default --session-
timeout
    Duration maxLifetime(Principal p);                                  // default none
    Duration revalidateInterval(Principal p);                           // default none; e.g. 5
min for Firebase → provider.revalidate
    boolean allowConcurrentSessions(Principal p);                       // false = logging in
elsewhere ends older sessions
}
public interface AuthenticationListener {
    default void loginSucceeded(AuthenticationEvent e) { }              // principal, provider,
remote address, methods
    default void loginFailed(AuthenticationEvent e) { }                 // username attempted (if
any), reason
    default void loggedOut(AuthenticationEvent e) { }                   // reason: USER, TIMEOUT,
REVALIDATION_FAILED, ADMIN, CONCURRENT
}
public interface UserDirectory {                                        // optional, for admin UIs
— not used by authentication itself
    Optional<UserInfo> findById(String id);  List<UserInfo> search(String query, int limit);
}
```

8a. Session store (sharing sessions between instances)

A session lives in the memory of the instance that created it: the `SessionManager` table, keyed by the `WBSESSION` cookie, holds the `HttpSession` with its CSRF token, `Authentication` and `UISession`. A second instance — or the same one after a restart — does not know the cookie and shows the login page. The optional **`SessionStore`** SPI (`org.webbeans.auth.spi`) makes the *authenticated* part of a session durable and shared:

```
public interface SessionStore {
    void put(StoredSession s) throws IOException;                        // on login
    StoredSession get(String sessionId) throws IOException;               // null = unknown or expired
    void touch(String id, Instant lastActivity, Instant expiresAt) throws IOException;   // ≤ 1 write
/ 10 s / session
    void remove(String sessionId) throws IOException;                     // logout, timeout, admin —
not on shutdown
    Collection<StoredSession> list() throws IOException;                  // admin views, concurrent-
session policy
}
// StoredSession: id, CSRF token, Authentication (principal, methods, expiry, secrets), locale, remote
address,
// node id, created / lastActivity / expiresAt; toJson() / fromJson(Map)
```

Semantics in the `SessionManager`:

- `create` (login) → `put`. The stored `expiresAt` is the earliest of idle timeout, maximum lifetime and the authentication's own expiry, computed from the `SessionPolicy`; the sweeper `touch`es it when the session was active since the last write.
- `get(cookie)` with no session in memory → `store.get`: if present and unexpired the instance **adopts** it — an `HttpSession` with the *same id and CSRF token* and a fresh `UISession` for the stored `Authentication` — and logs `adopted from the session store (created on <node>)`. The next `GET /` builds a new UI; the user is signed in but starts with the default window layout for that tab (per-user layout files are on the data dir, so they come back when the data dir is shared).
- `invalidate` → `remove`, except for `SHUTDOWN`: a stopping node leaves its sessions for the others, which is what makes rolling restarts invisible to users.
- Store failures are logged and ignored: the instance keeps working from memory; the degradation is "re-login after failover", never "cannot sign in".
- `Authentication.getSecrets()` *is* stored (an OIDC refresh token must survive the failover to be useful), so the store must be as private as the server's memory: a directory readable by the server user only, or an authenticated database/Redis.

The shipped example is **`FileSessionStore`** (`webbeans-server`): one JSON file per session in a directory every instance mounts (NFS, SMB, a shared volume), atomic writes, tolerant reads, expired files pruned once a minute. Enable it with `--session-store /shared/wb/sessions` (and `--node-id app-1`, default the host name) or `new ServerConfig().sessionStore(new FileSessionStore(dir))`, or register any implementation with `@ServiceProvider(service = SessionStore.class)`. A JDBC or Redis store is the same five methods with another back end; `StoredSession.toJson()`/`fromJson()` give it the serialisation. `SessionStoreTest` runs two `SessionManager`s over one store.

9. HTTP surface (fixed, provider-independent)

Endpoint	Purpose
`GET /login`	Login page (`LoginPage.render`), `?return=` preserved
`POST /login`	Form credentials (`PasswordCredentials` or a custom form) or JSON `{"type": "...", ...}`; CSRF checked
`POST /auth/token`	`TokenCredentials` from a browser SDK; also refresh on an existing session
`GET /auth/<id>/start`, `GET|POST /auth/<id>/callback`	Redirect flows
`POST /auth/challenge`	`ChallengeResponse` for multi-step logins
`POST /logout`	Ends the session, calls `provider.logout`, redirects to `/login` or the provider's end-session URL
`GET /auth/me`	JSON of the current principal (for custom front-ends)
`GET /wb/health`	`{"status":"ok","node":…,"sessions":n}` without a session — for load-balancer health checks (server module)

Anything beyond this (device login, magic links) registers an `AuthHttpHandler` under `/auth/<id>/...`:

```
public interface AuthHttpHandler { String getPathPrefix(); void handle(AuthHttpRequest req,
AuthHttpResponse res, Authenticator auth); }
```

`AuthHttpRequest`/`Response` are the platform's small HTTP abstractions (the same ones the Jetty and JDK transports implement), so handlers do not depend on the container.

10. Providers shipped with the platform

Provider	Credentials	Notes
`PropertiesAuthenticationProvider` (`local`)	`PasswordCredentials`	`<datadir>/system/users.properties`, PBKDF2-SHA256; `PasswordTool` CLI; the default for new apps
`HeaderAuthenticationProvider`	`HeaderCredentials`	`X-Remote-User`/`X-Remote-Groups` (configurable) from trusted proxies
`JwtAuthenticationProvider` (`jwt`)	`TokenCredentials`	RS256/ES256 via JWKS URL or static keys; issuer/audience checks; claim → attribute mapping. Base for the next two
`FirebaseAuthenticationProvider` (`firebase`)	`TokenCredentials`	Verifies Firebase ID tokens (Google's X.509 certs, `aud` = project id, `iss` = `https://securetoken.google.com/<project>`); `LoginMethod.script` with the Firebase Web SDK and FirebaseUI; custom claims → roles via `RoleMapper`; refresh every 55 min
`OidcAuthenticationProvider`	`CallbackCredentials`	Authorization-code + PKCE against any OIDC discovery URL (Google, Microsoft Entra, Keycloak, Auth0); ID-token verification through the JWT provider; end-session on logout
`LdapAuthenticationProvider`	`PasswordCredentials`	bind + group search (JNDI, JDK only)

Each is configured from `<datadir>/system/auth/<id>.properties` (or programmatically), and enabled simply by being on the module path — the `Authenticator` lists whatever providers the Lookup contains.

11. Examples

Firebase — add the `webbeans-auth-firebase` module, `system/auth/firebase.properties`:

```
projectId=acme-prod
webApiKey=AIza...                # for the login script
authDomain=acme-prod.firebaseapp.com
roleClaim=roles                  # custom claim holding an array of roles (set with the Admin SDK)
```

Nothing else changes; `@TopComponent.Registration(roles = {"admin"})` now follows the Firebase custom claim.

Own user table — one class:

```
@ServiceProvider(service = AuthenticationProvider.class, position = 50)
public final class AcmeUsersProvider implements AuthenticationProvider {
    public String getId() { return "acme"; }
    public String getDisplayName() { return "#CTL_AcmeLogin"; }
    public boolean supports(Class<? extends Credentials> t) { return
PasswordCredentials.class.isAssignableFrom(t); }
    public LoginMethod getLoginMethod() { return LoginMethod.form(); }
    public AuthenticationResult authenticate(Credentials c, AuthContext ctx) {
        PasswordCredentials pc = (PasswordCredentials) c;
        AcmeUser u = users.find(pc.getUsername());
        if (u == null || !PasswordHash.verify(pc.getPassword(), u.hash)) return
AuthenticationResult.failure("#ERR_Login");
        if (u.totpEnabled) return AuthenticationResult.challenge(Challenge.of("totp",
LoginField.text("code", "#LBL_Code")));
        return
AuthenticationResult.success(Principal.builder(u.id).name(u.login).displayName(u.fullName).roles(u.rol
es).build()).method("password");
    }
}
```

Existing SSO that sets a JWT cookie — register `JwtAuthenticationProvider` with `tokenSource=cookie:SSO_TOKEN` and a `RoleMapper` for its claims; no login page is shown because the session is created silently from the first request.

Reverse proxy — `--trusted-proxies 10.0.0.0/8`, `HeaderAuthenticationProvider` enabled; the proxy does the rest.

12. Security notes

- Providers never see or set cookies; sessions, CSRF and fixation protection are platform code.
- `AuthenticationResult.failure` messages are generic by convention (`#ERR_Login`); listeners get the detail.
- Redirect flows: `state` single-use and bound to the flow state; PKCE always on; callback only accepted over the configured `--public-url`.
- Token flows: clock skew ±60 s, `exp`/`nbf`/`iss`/`aud` mandatory, keys cached with the provider's cache headers, algorithm allow-list (no `none`, no HMAC with public keys).
- `Authentication.getSecrets()` is server memory only; `UISession` serialisation (if ever added) skips it.
- Step-up: `Authenticator.reauthenticate` + `Authentication.getMethods()` let an action demand a fresh `password` or `mfa` method before a sensitive operation.

13. Implementation plan

- Step 2 (now): `webbeans-auth` API, `Authenticator` implementation, `PropertiesAuthenticationProvider`, `HeaderAuthenticationProvider`, default `LoginPage` (form + redirect buttons + script methods), `SessionPolicy` defaults, listeners, throttle, the HTTP endpoints in the server.
- Step 2b (with Jetty): `JwtAuthenticationProvider` (JWKS fetch needs outbound HTTP), `FirebaseAuthenticationProvider`, `OidcAuthenticationProvider`, `LdapAuthenticationProvider`, each as its own module so applications ship only what they use.

14. Load balancing and multi-instance deployment

The constraint. A WebBeans session is stateful by design: the component tree, window layout, open editors and the registry views of each signed-in user live in the JVM they signed in on, and a tab talks to that JVM over one WebSocket (or long-poll channel). This is the same shape as the NetBeans Platform it mirrors, or any server-side-UI framework (Vaadin, ZK): the session *is* the application. Consequently instances can run on different machines and users can be spread across them, but the balancing must be **sticky per session**, not per request, and the UI state of a session does not move between instances.

What works out of the box.

- *Sticky reverse proxy.* nginx, HAProxy, Traefik, an AWS ALB: stickiness on the `WBSESSION` cookie (or the balancer's own cookie), WebSocket upgrade forwarding, `X-Forwarded-For` / `X-Forwarded-Proto` set, and each instance started with `--trusted-proxy <lb-ip>` so client addresses, HTTPS detection and secure cookies are right. Every request of a tab — bootstrap page, `/wb/ws`, `/wb/poll`, uploads, `/wb/blob`, `/wb/tiles`, `/wb/download` — carries the cookie, so one user's traffic lands on one instance. `balance leastconn` suits long-lived WebSocket connections better than round-robin.
- *Independent instances.* Each has its own Jetty, session table, executor pool and bundle; nothing is shared in memory, so throughput scales linearly and a failure is isolated. `GET /wb/health` answers without a session for the balancer's checks (`node` says which instance answered).
- *Shared per-user state on disk.* `<datadir>/users/<u>/` (layouts, preferences, module files) and the `PropertiesAuthenticationProvider` user file should be on a shared data dir (NFS/SMB) or synced, so a user who lands on a different instance next week finds their layout and the accounts are not per instance; an external identity provider (LDAP, OIDC, headers from the proxy) makes the account question disappear. Tile caches and upload temp files can stay local.
- *Login survives failover* with a `SessionStore` (§8a): when an instance goes away, the next request of its users hits another instance, which adopts the session from the store and builds a fresh UI — no login page. Without a store, users of a failed instance re-authenticate; their UI state is gone in both cases.

What stays per instance, and the consequences.

1. *UI state.* Open windows and editors of a session exist only in the JVM that built them. Failover gives the user a signed-in but freshly laid-out tab (the saved layout is restored from the shared data dir). Rolling restarts therefore cost users their unsaved editor contents; drain an instance (stop sending it new sessions, wait for `sessions` in `/wb/health` to fall) before stopping it when that matters.
2. *Cross-instance events.* `UISession.publish`, the session bus, `ServerSessionDirectory`, the admin "sessions" view and the concurrent-sessions policy see the local instance only. Anything users must see from each other — a shared model being edited, server-wide notifications, "kick this user everywhere" — goes through the application's own backend (database, message broker) and is applied on each instance's executor; `SessionStore.list()` gives admin views the global session list.
3. *Capacity is memory, not CPU.* A session costs roughly its open component tree (tens of KB to a few MB with large tables, graphs or scenes) plus the delta history kept for reconnects; size

instances by concurrent sessions and set `--session-timeout` so abandoned tabs are reclaimed. One backend pool (`ServerConfig.backendThreads`) serves every session's tasks on an instance.

Recommended shape.

```
                ┌─ app-1  java -jar app.jar --datadir /shared/wb --session-store /shared/wb/sessions
--node-id app-1 --trusted-proxy 10.0.0.5 --port 8080
users ─► HAProxy ┼─ app-2  (same, --node-id app-2)
   TLS, sticky,  └─ app-3  (same, --node-id app-3)
   WS upgrade
                /shared/wb   users/, system/ (accounts), sessions/ on NFS or a shared volume; auth
via LDAP/OIDC or the proxy
```

HAProxy:

```
frontend https
    bind :443 ssl crt /etc/haproxy/app.pem
    http-request set-header X-Forwarded-Proto https
    default_backend webbeans

backend webbeans
    balance leastconn
    cookie WBSTICKY insert indirect nocache httponly secure
    option forwardfor
    option httpchk GET /wb/health
    server app1 10.0.1.11:8080 check cookie app1
    server app2 10.0.1.12:8080 check cookie app2
    server app3 10.0.1.13:8080 check cookie app3
```

nginx (`ip_hash` is the stickiness available without the commercial `sticky` module; a hash of the `WBSESSION` cookie with `hash $cookie_WBSESSION consistent` is the better choice when cookies are present):

```
upstream webbeans { hash $cookie_WBSESSION consistent; server 10.0.1.11:8080; server 10.0.1.12:8080;
server 10.0.1.13:8080; }
server {
    listen 443 ssl; # certificates …
    location / {
        proxy_pass http://webbeans;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade";
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-
Proto https;
        proxy_read_timeout 1h;   # idle WebSockets
    }
}
```

Alternatives, briefly. *Vertical first*: one instance serves hundreds of concurrent sessions on a modest machine, so a single bigger JVM behind TLS is often the simplest answer. *Per-tenant instances*: route by host name or path prefix (`--context /tenant-a`) rather than by load, which also keeps tenants' data dirs apart. *Active–passive*: one instance plus a standby with the shared data dir and session store — users re-attach after a failover without logging in. Kubernetes: a `Deployment` with session affinity (`sessionAffinity: ClientIP` on the Service, or cookie affinity on the ingress), a `ReadWriteMany` volume for the data dir and session store, `/wb/health` as readiness and liveness probe, `terminationGracePeriodSeconds` long enough to drain.